

DATA620_Assignment_Four_High_Frequency_Words

March 26, 2026

1 Jupyter Notebook: High Frequency Words (Bible Corpus)

1.1 Convert the PDF into a text file

```
[8]: !{sys.executable} -m pip install pymupdf

import fitz # PyMuPDF

doc = fitz.open(r"C:\Users\zengo\OneDrive\Magic Briefcase\School of_
↳Professional Studies\DATA 620 Web Analytics\CSB_Pew_Bible_2nd_Printing.pdf")

text = ""

for page in doc:
    text += page.get_text()

print("Characters:", len(text))

with open("bible.txt", "w", encoding="utf-8") as f:
    f.write(text)
```

Requirement already satisfied: pymupdf in c:\users\zengo\anaconda3\lib\site-packages (1.27.2.2)
Characters: 5142103

```
[9]: ## SECTION 1 - Load and Prepare the Corpus
```

```
[11]: # =====
# SECTION 1: LOAD BIBLE TEXT
# =====

import re
from collections import Counter
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

file_path = "bible.txt"
```

```
with open(file_path, "r", encoding="utf-8") as f:
    text = f.read()

print("Total characters:", len(text))
```

Total characters: 5142103

1.2 SECTION 2 — Clean and Tokenize Text

```
[12]: # =====
# SECTION 2: CLEAN TEXT
# =====

# Lowercase
text = text.lower()

# Remove punctuation and numbers
text = re.sub(r'[^a-z\s]', '', text)

# Tokenize into words
words = text.split()

print("Total words:", len(words))
```

Total words: 796879

1.3 SECTION 3 — Unique Words Count

```
[13]: # =====
# SECTION 3: UNIQUE WORDS
# =====

unique_words = set(words)

print("Total unique words:", len(unique_words))
```

Total unique words: 17869

1.4 SECTION 4 — Word Frequency Analysis

```
[14]: # =====
# SECTION 4: WORD FREQUENCIES
# =====

word_counts = Counter(words)

# Convert to DataFrame
```

```
df = pd.DataFrame(word_counts.items(), columns=["word", "count"])

# Sort by frequency
df = df.sort_values(by="count", ascending=False)

df.head(10)
```

```
[14]:
```

	word	count
0	the	57655
33	and	29335
19	of	25315
32	to	21774
509	you	14234
53	a	12589
23	in	11870
679	will	10476
587	he	9794
949	i	9296

1.5 SECTION 5 — Words Representing 50% of Corpus

```
[15]: # =====
# SECTION 5: 50% COVERAGE
# =====

total_words = len(words)
cumulative = df['count'].cumsum()

# Find how many words make up 50%
half_point = total_words * 0.5

num_words_half = (cumulative <= half_point).sum()

print("Number of words accounting for 50%:", num_words_half)
```

Number of words accounting for 50%: 55

1.6 SECTION 6 — Top 200 Words

```
[19]: # =====
# SECTION 6: TOP 200 WORDS
# =====

top_200 = df.head(200).copy()

top_200.head()
```

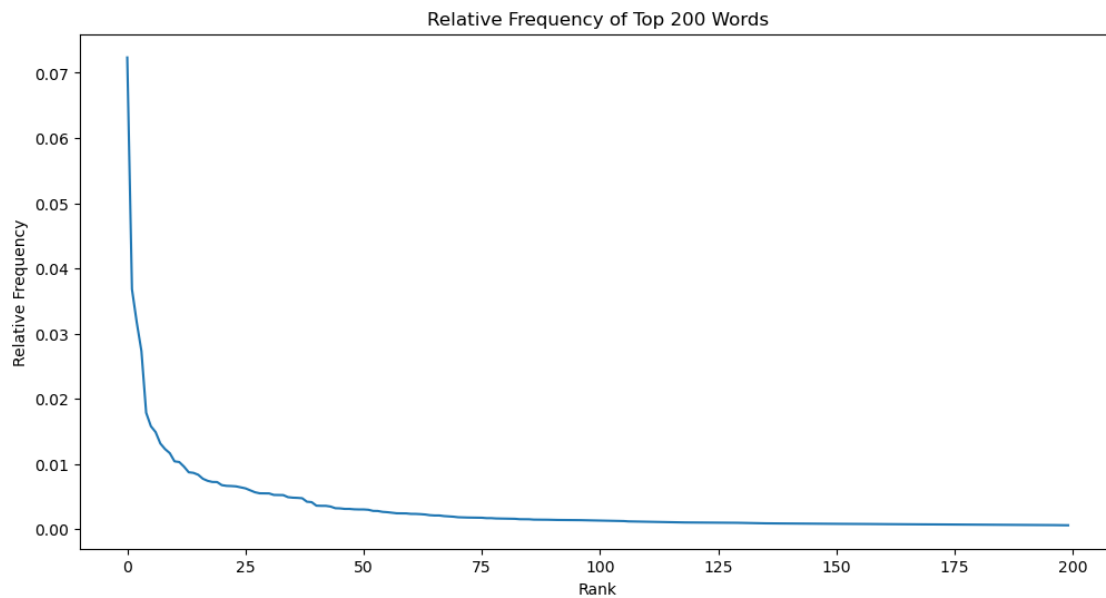
```
[19]:      word  count
      0    the  57655
      33   and  29335
      19    of  25315
      32    to  21774
      509  you  14234
```

1.7 SECTION 7 — Plot Relative Frequency

```
[20]: # =====
# SECTION 7: RELATIVE FREQUENCY GRAPH
# =====

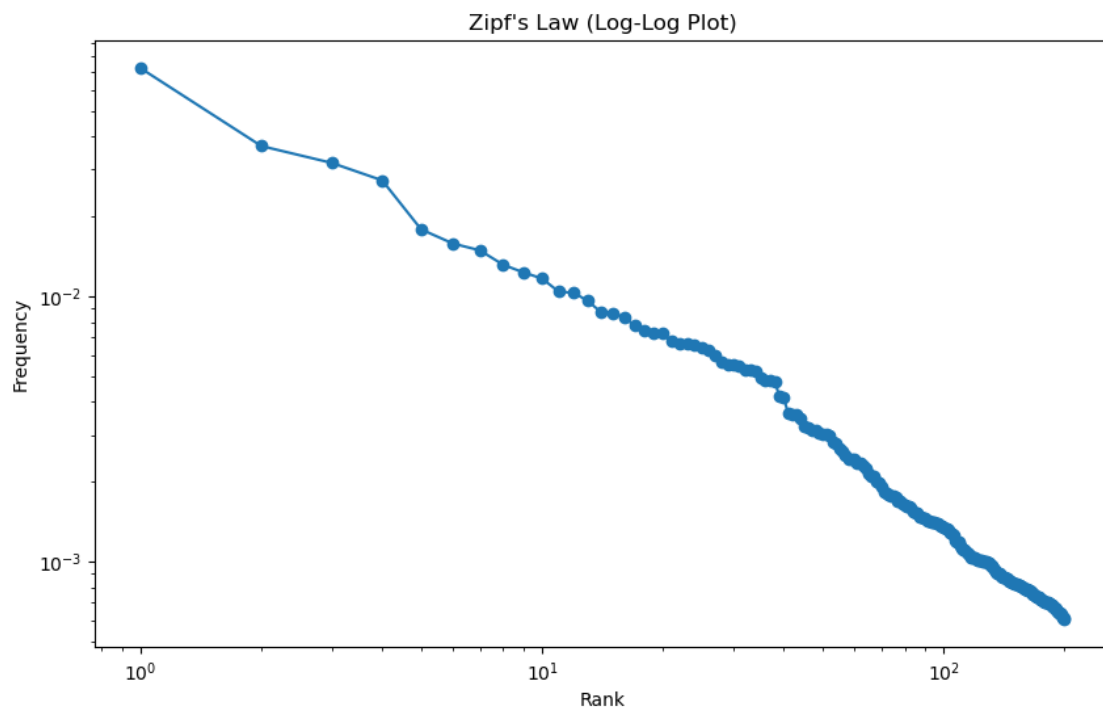
top_200['relative_freq'] = top_200['count'] / total_words

plt.figure(figsize=(12,6))
plt.plot(top_200['relative_freq'].values)
plt.title("Relative Frequency of Top 200 Words")
plt.xlabel("Rank")
plt.ylabel("Relative Frequency")
plt.show()
```



1.8 SECTION 8 — Zipf's Law Analysis

```
[21]: # =====  
# SECTION 8: ZIPF'S LAW  
# =====  
  
ranks = np.arange(1, 201)  
frequencies = top_200['relative_freq'].values  
  
plt.figure(figsize=(10,6))  
plt.loglog(ranks, frequencies, marker="o")  
plt.title("Zipf's Law (Log-Log Plot)")  
plt.xlabel("Rank")  
plt.ylabel("Frequency")  
plt.show()
```



1.9 SECTION 9 — Interpretation (MARKDOWN CELL)

Zipf's Law: The plot shows that word frequency decreases roughly inversely with rank. This suggests the Bible corpus approximately follows Zipf's Law, where:

$$1 \quad \text{frequency} \quad \text{rank} \quad 1$$

Observations:

High-frequency words include function words like the, and, of, to A small number of words dominate

the corpus Long-tail distribution exists (many rare words)

1.10 SECTION 10 — Reflection (MARKDOWN)

[]:

[]: